

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 9

дисциплина: Архитектура компьютера _

Студент: Ромеро Гаспар Фернандо Алексей

Группа: НКАбд-02-25

МОСКВА

2025 г.

Оглавление

Цель заботы

Теоретическое введение

2.1 Понятие об отладке

2.2 Методы отладки

Выполнения лабораторной работы

3.1 Реализация подпрограмм в NASM

3.2 Отладка программ с помощью GDB

3.3 Аргументы командной строки в GDB

Выполнение самостоятельной работы

4.1 Преобразование программы

4.2 Альтернативная реализация

Выводы

Список литературы

Цель работы

Приобретение навыков написания программ с использованием подпрограмм.
Знакомство с методами отладки при помощи GDB и его основными возможностями.

Теоретическое введение

2.1 Понятие об отладке

Отладка — это систематический процесс выявления, анализа и исправления ошибок (багов) в компьютерной программе. Она выполняется с использованием специализированных инструментов, называемых отладчиками (debuggers), которые позволяют выполнять код пошагово, инспектировать состояние памяти, регистров процессора и переменных во время выполнения. Основные цели включают:

1. Локализацию первопричины сбоя,
2. Проверку потока выполнения программы,
3. Валидацию состояния данных в определённые моменты времени и
4. Исправление нежелательного поведения без внесения новых ошибок.

Существует множество методов отладки, среди наиболее часто используемых:

- **Интерактивная отладка:** Использование отладчика для выполнения программы с точками останова (breakpoints), проверки стека вызовов (call stack) и изменения значений в памяти.
- **Логирование и трассировка:** Вставка регистрационных сообщений в код для отслеживания выполнения программы и состояния переменных.
- **Посмертный анализ:** Изучение дампов памяти (core dumps) после серьёзного сбоя.
- **Удалённая отладка:** Подключение к программе, выполняющейся в другой системе или среде.

При отладке низкоуровневого кода или двоичных файлов без символов отладки используются специальные методы:

- **Прямая проверка регистров и памяти:** Инспектирование регистров, таких как EIP/RIP (указатель инструкций), ESP/RSP (указатель стека) и EFLAGS/RFLAGS.
- **Трассировка стека:** Анализ кадров стека (stack frames) для восстановления последовательности вызовов функций.
- **Аппаратные точки останова:** Прерывания, генерируемые процессором (INT 3 в x86),

которые останавливают выполнение по определённому адресу.

- **Горячая модификация:** Изменение инструкций или значений в памяти во время выполнения для проверки исправлений.

Такие инструменты, как GDB, WinDbg и radare2, позволяют проводить этот низкоуровневый анализ, что критически важно для реверс-инжиниринга, анализа вредоносного ПО и разработки драйверов.



2.2 Методы отладки

Методы интерактивной отладки предполагают контролируемое выполнение программы с помощью специализированных инструментов. Основные техники включают:

- **точки останова (breakpoints)**, которые приостанавливают выполнение на определённых строках.
- **пошаговое выполнение**, позволяющее продвигаться по одной инструкции (step into) или по одной функции (step over).
- **инспекцию переменных в реальном времени** с помощью окон наблюдения (watch windows).
- **анализ стека вызовов (call stack)** для отслеживания последовательности выполнения функций.

Эти инструменты интегрированы в IDE, такие как Visual Studio и Eclipse, а также в отладчики командной строки, такие как GDB и LLDB, что позволяет выявлять логические ошибки, ошибки потока управления и неверные значения.

Методы неинтерактивной отладки, такие как логирование и трассировка, предполагают

вставку инструкций вывода (например, ``printf``, ``console.log``) в стратегические точки кода для регистрации значений переменных, сообщений о состоянии и потоков выполнения. Эта техника особенно полезна в средах, где нет доступа к интерактивному отладчику, таких как встроенные системы, рабочие серверы или мобильные приложения. Инструментирование кода с помощью библиотек структурированного логирования позволяет создавать записи, классифицированные по уровням серьёзности (debug, info, error), и облегчает их последующий анализ с помощью инструментов агрегации и поиска.

Для сложных ошибок, таких как утечки памяти, повреждение данных или состояния гонки (race conditions), используются продвинутые методы:

- **анализ памяти** с помощью инструментов вроде Valgrind, AddressSanitizer или Dr. Memory.
- **отладка потоков (thread debugging)** для выявления взаимных блокировок (deadlocks) и состояний гонки.
- **обратная отладка (time-travel debugging)**, позволяющая выполнять код как вперёд, так и назад.
- **анализ дампов памяти (core dumps)** после сбоя программы.

При разработке драйверов и операционных систем также применяются отладчики уровня ядра, такие как WinDbg/KD, для анализа системных сбоев.

Выполнения лабораторной работы

3.1. Реализация подпрограмм в NASM

Создайте программу lab09-1.asm для вычисления комплексного выражения $f(g(x))$, где $f(x) = 2x + 7$ и $g(x) = 3x - 1$.

Для реализации этой задачи применяется принцип модульности с использованием вложенных подпрограмм. Это позволяет разделить вычислительную логику на независимые блоки. Основная программа вызывает подпрограмму ``_calcul``, отвечающую за внешнюю функцию $f(x)$. В свою очередь, ``_calcul`` вызывает вложенную подпрограмму ``_subcalcul``

для предварительного вычисления значения внутренней функции ``$g(x)$``. Передача результатов вычислений между подпрограммами осуществляется с помощью регистра EAX, что является стандартным соглашением для возврата значений.

```
fernando@ubuntu25-10:~$ mkdir ~/work/arch-oc/lab09
mkdir: No such file or directory
fernando@ubuntu25-10:~$ mkdir ~/work/arch-pc/lab09
fernando@ubuntu25-10:~$ cd ~/work/arch-pc/lab09
fernando@ubuntu25-10:~/work/arch-pc/lab09$ touch lab9-1.asm
fernando@ubuntu25-10:~/work/arch-pc/lab09$ nano lab9-1.asm
fernando@ubuntu25-10:~/work/arch-pc/lab09$ nasm -f elf lab9-1.asm
fernando@ubuntu25-10:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab9-1 lab9-1.o
fernando@ubuntu25-10:~/work/arch-pc/lab09$ ./lab9-1
Введите x: 4
2x+7=15
```

Вы скомпилировали файл и проверили его корректность, используя тестовое значение $x = 4$. Правильность выполнения проверялась шаг за шагом:

- Ввод: пользователь вводит $x = 4$.
- Преобразование: Программа преобразует строку «4» в число 4.
- Расчет по формуле: $f(4) = 2 \cdot 4 + 7 = 8 + 7 = 15$
- Вывод результата: Программа отображает «2x+7=15».

Программа выдала ожидаемый результат 15, подтверждая правильную работу функции `_calcul`, инструкции `mul` для умножения, инструкции `add` для сложения и инструкции `ret` для возврата из подпрограммы.

3.2. Отладка программ с помощью GDB

Создайте файл с именем `lab09-2.asm`, который отображает сообщение "Hello World", и скомпилируйте его с опцией `-g`. Эта опция крайне важна, поскольку она добавляет отладочную информацию к исполняемому файлу, позволяя отладчику сравнивать машинный код с исходным кодом программы. Затем загрузите исполняемый файл в отладчик `gdb`.

```
fernando@ubuntu25-10:~/work/arch-pc/lab09$ nano lab9-2.asm
fernando@ubuntu25-10:~/work/arch-pc/lab09$ nasm -f elf -g lab9-2.asm -o lab9-2.o
fernando@ubuntu25-10:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab9-2 lab9-2.o
fernando@ubuntu25-10:~/work/arch-pc/lab09$ gdb lab9-2
GNU gdb (Ubuntu 16.3-1ubuntu2) 16.3
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab9-2...
(gdb)
```

При работе с отладчиком выполните несколько шагов для анализа состояния процессора:

1. Запустите программу с помощью команды ``run``.
2. Установите точку останова в метке ``_start`` с помощью команды ``break _start``, чтобы прервать выполнение до первой инструкции.
3. Включите режим дизассемблирования и переключитесь на синтаксис Intel с помощью команды ``set disassembly-flavor intel``. Это упростит чтение кода, поскольку порядок операндов станет привычным (значение, исходный код), как в исходном коде NASM, в отличие от стандартного синтаксиса AT&T для GDB.
4. Активируйте псевдографический интерфейс с помощью команд ``layout asm`` и ``layout regs``. Это разделит окно терминала на разделы, отображающие текущий ассемблерный код, состояние регистров и командную строку.

```
(gdb) run
Starting program: /home/fernando/work/arch-pc/lab09/lab9-2

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Hello, word!
[Inferior 1 (process 5469) exited normally]
(gdb)
```

```
(gdb) break _start
Breakpoint 1 at 0x8049000: file lab9-2.asm, line 12.
(gdb) run
Starting program: /home/fernando/work/arch-pc/lab09/lab9-2

Breakpoint 1, _start () at lab9-2.asm:12
12      mov     eax, 4
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08049000 <+0>:    mov     $0x4,%eax
0x08049005 <+5>:    mov     $0x1,%ebx
0x0804900a <+10>:   mov     $0x804a000,%ecx
0x0804900f <+15>:   mov     $0x8,%edx
0x08049014 <+20>:   int     $0x80
0x08049016 <+22>:   mov     $0x4,%eax
0x0804901b <+27>:   mov     $0x1,%ebx
0x08049020 <+32>:   mov     $0x804a008,%ecx
0x08049025 <+37>:   mov     $0x6,%edx
0x0804902a <+42>:   int     $0x80
0x0804902c <+44>:   mov     $0x1,%eax
0x08049031 <+49>:   mov     $0x0,%ebx
0x08049036 <+54>:   int     $0x80
End of assembler dump.
(gdb) set disassembly-flavor intel
No symbol "disassembly" in current context.
(gdb) set disassembly-flavor intel
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08049000 <+0>:    mov     eax,0x4
0x08049005 <+5>:    mov     ebx,0x1
0x0804900a <+10>:   mov     ecx,0x804a000
0x0804900f <+15>:   mov     edx,0x8
0x08049014 <+20>:   int     0x80
0x08049016 <+22>:   mov     eax,0x4
0x0804901b <+27>:   mov     ebx,0x1
0x08049020 <+32>:   mov     ecx,0x804a008
0x08049025 <+37>:   mov     edx,0x6
0x0804902a <+42>:   int     0x80
0x0804902c <+44>:   mov     eax,0x1
0x08049031 <+49>:   mov     ebx,0x0
0x08049036 <+54>:   int     0x80
End of assembler dump.
```

Используйте команду `si` (пошаговое выполнение инструкции) для пошагового выполнения, наблюдая за изменением значений регистров `eax`, `ebx`, `ecx` и `edx` после каждой

инструкции `mov` и `int`. Также следите за регистром `eip` (указатель инструкций), который указывает на адрес следующей инструкции, подлежащей выполнению.

```
fernando@ubuntu25-10: ~/work/arch-pc/lab09 — gdb lab9-2
~/work/arch-pc/lab09

Register group: general
eax      0x4      4      ecx      0x0      0
edx      0x0      0      ebx      0x0      0
esp      0xffffcfff 0xffffcfff  ebp      0x0      0x0
esi      0x0      0      edi      0x0      0
eip      0x8049005 0x8049005 <_start+5>  eflags    0x10202    [ IF RF ]
cs       0x23     35     ss       0x2b     43
ds       0x2b     43     es       0x2b     43
fs       0x0      0      gs       0x0      0

B+ 0x8049000 <_start> mov eax,0x4
>0x8049005 <_start+5> mov ebx,0x1
0x804900a <_start+10> mov ecx,0x804a000
0x804900f <_start+15> mov edx,0x8
0x8049014 <_start+20> int 0x80
0x8049016 <_start+22> mov eax,0x4
0x804901b <_start+27> mov ebx,0x1
0x8049020 <_start+32> mov ecx,0x804a008
0x8049025 <_start+37> mov edx,0x6
0x804902a <_start+42> int 0x80
0x804902c <_start+44> mov eax,0x1
0x8049031 <_start+49> mov ebx,0x0

native process 5482 (asm) In: _start L13 PC: 0x8049005
eax      0x4      4
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffcfff 0xffffcfff
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8049005 0x8049005 <_start+5>
eflags    0x10202    [ IF RF ]
cs       0x23     35
ss       0x2b     43
ds       0x2b     43
es       0x2b     43
--Type <RET> for more, q to quit, c to continue without paging--
```

3.3. Аргументы командной строки в GDB

Для изучения передачи параметров я скопировал программу из предыдущей лабораторной работы в файл `lab09-3.asm` и запустил её в отладчике с аргументами командной строки. Особое внимание я уделю анализу стека.

Изучите вершину стека с помощью команды ``x/x $esp`` (просмотр памяти по адресу

регистра указателя стека) и подтвердите следующее:

- На вершине стека (адрес по адресу ``$esp``) хранится количество аргументов (`argc`).
- Следующая ячейка памяти, ``$esp + 4``, содержит адрес имени программы.
- Далее, по адресам ``$esp + 8``, ``$esp + 12`` и так далее, хранятся указатели на сами строки аргументов.

Шаг смещения составляет 4 байта, поскольку мы работаем с 32-битной архитектурой, где каждый адрес памяти занимает ровно 4 байта (32 бита). Это наблюдение подтверждает теоретическое понимание того, как операционная система передает данные программе при запуске.

```
Fernando@ubuntu25-10:~/work/arch-pc/lab09$ nasm -f elf -g -l lab9-3.lst lab9-3.asm
Fernando@ubuntu25-10:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab9-3 lab9-3.o
Fernando@ubuntu25-10:~/work/arch-pc/lab09$ gdb --args lab9-3 argument1 argument2 'argument3'
GNU gdb (Ubuntu 16.3-1ubuntu2) 16.3
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab9-3...
(gdb) █
```

```

(gdb) b _start
Breakpoint 1 at 0x80490e8: file lab9-3.asm, line 7.
(gdb) run
Starting program: /home/fernando/work/arch-pc/lab09/lab9-3 argument1 argument2 argument3

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.

x/x %esp

Breakpoint 1, _start () at lab9-3.asm:7
7          pop ecx          ; Извлекаем из стека в `ecx` количество
(gdb)
(gdb) x/x %esp
A syntax error in expression, near `%esp'.
(gdb) x/x $esp
0xffffcfc0: 0x00000004
(gdb) x/8x $esp
0xffffcfc0: 0x00000004 0xffffd1a8 0xffffd1d1 0xffffd1db
0xffffcfd0: 0xffffd1e5 0x00000000 0xffffd1ef 0xffffd1ff
(gdb) x/s *(int*)($esp + 4)
0xffffd1a8: "/home/fernando/work/arch-pc/lab09/lab9-3"
(gdb) x/s *(int*)($esp + 8)
0xffffd1d1: "argument1"
(gdb) x/s *(int*)($esp + 12)
0xffffd1db: "argument2"
(gdb) x/s *(int*)($esp + 16)
0xffffd1e5: "argument3"
(gdb) x/s *(int*)($esp + 20)
0x0: <error: Cannot access memory at address 0x0>
(gdb)

```

Выполнение самостоятельной работы

4.1 Преобразование программы

Напишите программу lab8-2.asm, которая демонстрирует, как получить и вывести все аргументы, переданные ей в начале. Сначала программа извлекает из стека количество аргументов (argc) и имя программы (argv[0]). Затем в цикле она последовательно извлекает и выводит оставшиеся аргументы.

```

fernando@ubuntu25-10:~/work/arch-pc/lab09$ nasm -f elf lab9-ind.asm
fernando@ubuntu25-10:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab9-ind lab9-ind.o
fernando@ubuntu25-10:~/work/arch-pc/lab09$ ./lab9-ind 11 21 23 19
Результат суммы f(x) = 15x + 2: 1118

```

4.2 Альтернативная реализация

2. В листинге 9.3 приведена программа вычисления выражения $(3 + 2) * 4 + 5$. При запуске данная программа дает неверный результат. Проверьте это. С помощью отладчика GDB, анализируя изменения значений регистров, определите ошибку и исправьте ее.

```
fernando@ubuntu25-10:~/work/arch-pc/lab09$ nasm -f elf lab9-ind2.asm
fernando@ubuntu25-10:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab9-ind2 lab9-ind2.o
fernando@ubuntu25-10:~/work/arch-pc/lab09$ ./lab9-ind2
Результат: 10
```

```
fernando@ubuntu25-10:~/work/arch-pc/lab09$ gdb lab9-ind2
GNU gdb (Ubuntu 16.3-1ubuntu2) 16.3
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab9-ind2...
(No debugging symbols found in lab9-ind2)
(gdb) break _start
Breakpoint 1 at 0x80490e8
(gdb) run
Starting program: /home/fernando/work/arch-pc/lab09/lab9-ind2

This GDB supports auto-downloading debuginfo from the following URLs:
    <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.

Breakpoint 1, 0x080490e8 in _start ()
(gdb) display $eax
1: $eax = 0
(gdb) display $ebx
2: $ebx = 0
(gdb) display $ecx
3: $ecx = 0
(gdb) display $edx
4: $edx = 0
(gdb) display $edi
5: $edi = 0
```

```
(gdb) stepi
0x080490ed in _start ()
1: $eax = 0
2: $ebx = 3
3: $ecx = 0
4: $edx = 0
5: $edi = 0
(gdb) stepi
0x080490f2 in _start ()
1: $eax = 2
2: $ebx = 3
3: $ecx = 0
4: $edx = 0
5: $edi = 0
(gdb) stepi
0x080490f4 in _start ()
1: $eax = 2
2: $ebx = 5
3: $ecx = 0
4: $edx = 0
5: $edi = 0
(gdb) stepi
0x080490f9 in _start ()
1: $eax = 2
2: $ebx = 5
3: $ecx = 4
4: $edx = 0
5: $edi = 0
(gdb) stepi
0x080490fb in _start ()
1: $eax = 8
2: $ebx = 5
3: $ecx = 4
4: $edx = 0
5: $edi = 0
(gdb) stepi
0x080490fe in _start ()
1: $eax = 8
2: $ebx = 10
3: $ecx = 4
4: $edx = 0
5: $edi = 0
(gdb) █
```

```
fernando@ubuntu25-10:~/work/arch-pc/lab09$ nano lab9-ind2.asm
fernando@ubuntu25-10:~/work/arch-pc/lab09$ nasm -f elf lab9-ind2.asm
fernando@ubuntu25-10:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab9-ind2 lab9-ind2.o
fernando@ubuntu25-10:~/work/arch-pc/lab09$ ./lab9-ind2
Результат: 25
```

Выводы

В ходе лабораторной работы:

1. Мы научились форматировать повторяющиеся участки кода в виде подпрограмм, используя инструкции ``call`` и ``ret``.
2. Мы поняли, как работает стек при вызове процедур (хранение адреса возврата).
3. Мы освоили отладчик GDB: запуск, пошаговое выполнение (`if`), установка точек останова (`break`), просмотр регистров (информационные регистры) и памяти (`x`).
4. Мы научились использовать отладчик для обнаружения логических ошибок в ассемблерном коде.

Список литературы

1. IBM Documentation. (2022). *What is debugging?*
https://www.ibm.com/docs/es/SSYKE2_9.0.0/com.ibm.java.90.doc/user/debugging.html
2. Microsoft Learn. (2023). *Depuración para principiantes absolutos*.
<https://learn.microsoft.com/es-es/visualstudio/debugger/debugging-absolute-beginners>
3. GDB Documentation. (2023). *Debugging with GDB*.
<https://www.sourceware.org/gdb/documentation/>
4. Log4j. (2024). *Apache Log4j 2*. <https://logging.apache.org/log4j/2.x/>
5. Microsoft Learn. (2024). *Debugging techniques and tools*. <https://learn.microsoft.com/en-us/windows-hardware/drivers/debugger/>
6. Mozilla MDN. (2023). *Debugging JavaScript*. https://developer.mozilla.org/en-US/docs/Learn/JavaScript/First_steps/What_went_wrong
7. Oracle. (2023). *Debugging Java programs*.
<https://docs.oracle.com/javase/8/docs/technotes/tools/windows/jdb.html>
8. Valgrind. (2023). *Valgrind documentation*. <https://valgrind.org/docs/>
- 9.